

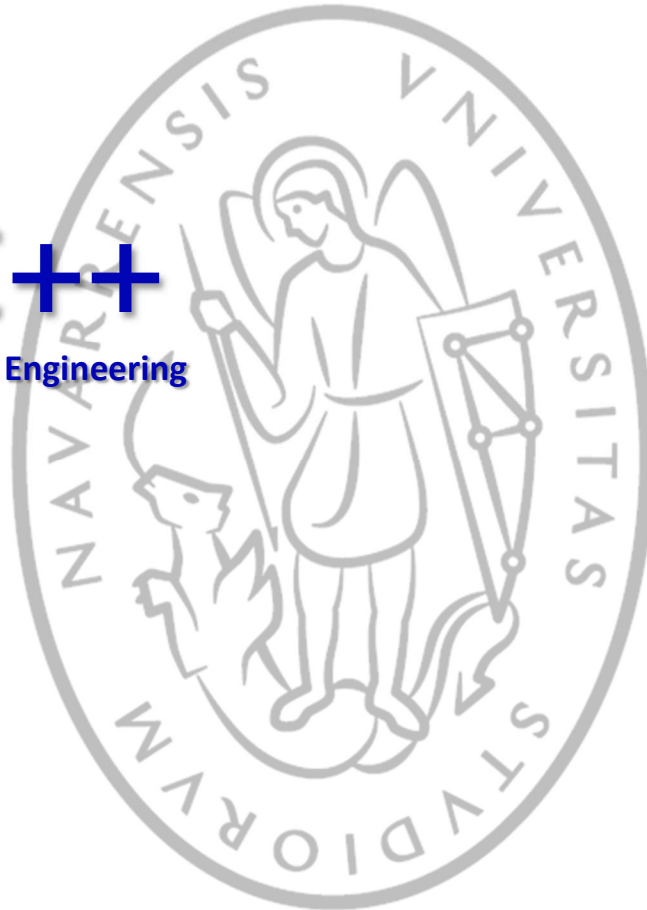


Tecnun  
Universidad  
de Navarra

ESCUELA DE INGENIERÍA  
INGENIARITZA ESKOLA  
SCHOOL OF ENGINEERING

## Examen Dic'25

**C++**  
For Engineering



**Informática II**

Dr. Paul Bustamante / Dr. Jesús Paredes



## Contenido

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>1. Introducción.....</b>                                           | <b>2</b>  |
| <b>2. Ejercicios.....</b>                                             | <b>3</b>  |
| 2.1. <i>Conversor Decimal a Hexadecimal y viceversa (2.0).....</i>    | <i>3</i>  |
| 2.2. <i>Polígonos irregulares con clases (2.5 pts.).....</i>          | <i>4</i>  |
| 2.3. <i>Sistema eléctrico con herencia (3.5 pts.).....</i>            | <i>5</i>  |
| 2.3.1. Opción 1: Mostrar fecha con distintos formatos .....           | 8         |
| 2.3.2. Opción 2: Leer nombres de fichero y mostrar por pantalla ..... | 8         |
| 2.3.3. Opción 3: Ordenar nombres leídos y mostrar por pantalla .....  | 9         |
| 2.3.4. Opción 4: Añadir central .....                                 | 9         |
| 2.3.5. Opción 5: Añadir mantenimiento.....                            | 11        |
| 2.3.6. Opción 6: Mostrar información.....                             | 12        |
| 2.3.7. Opción 7: Grabar datos en fichero .....                        | 13        |
| 2.3.8. Opción 8: Ajustar potencia de una central.....                 | 13        |
| 2.3.9. Opción 9: Ajustar potencia del sistema.....                    | 15        |
| 2.3.10. Opción 10: Salir .....                                        | 16        |
| <b>3. ANEXO A: COMPRIMIR CARPETAS .....</b>                           | <b>17</b> |

## 1. Introducción.

Puntos a tener en cuenta en el examen (si **no las hace**, no se podrá recoger el examen):

- Debe guardar los ejercicios en la carpeta **C:\TEMP**, así si se apaga el ordenador no perderás el trabajo.
- **Debe comprimir** cada carpeta y dejarla en **C:\Temp**, en formato **“.7z”** (ver anexo al final)
- **NO CERRAR LA SESION** al terminar el examen.
- **Si no tenemos los ficheros (.7z) no podremos corregir y tendrá “0” en el ejercicio.**

## 2. Ejercicios

### 2.1. Conversor Decimal a Hexadecimal y viceversa (2.0)

Este ejercicio consiste en hacer un programa que permita convertir un número decimal en hexadecimal y viceversa, el cual debe ser pedido por teclado.

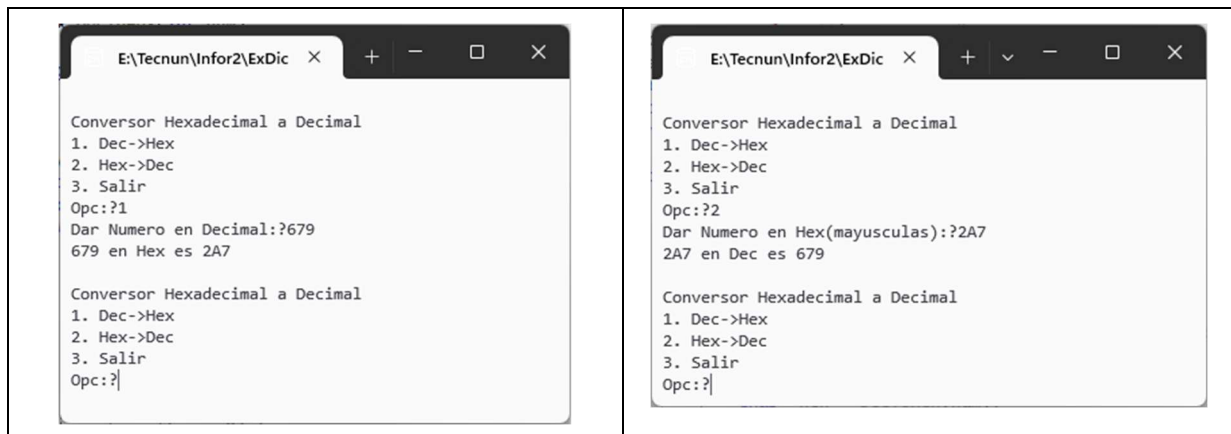
El sistema **hexadecimal** es un sistema de numeración posicional de base 16 que usa 16 símbolos (**0-9 y A-F**) para representar números de forma más compacta, siendo fundamental en informática para direcciones de memoria, códigos de color, etc.

|           |  |       |    |
|-----------|--|-------|----|
| 679       |  | 16    |    |
| 7         |  | 42    | 16 |
|           |  | (A)10 | 2  |
| Res = 2A7 |  |       |    |

Para convertir de **decimal** a **hexadecimal**, debe dividir el número entre 16 sucesivamente hasta que el resto sea menor que 16. Los números mayores o iguales a 10, deben ser sustituidos por letras, donde la A=10, B=11,.. F=15. Vea el ejemplo de la figura.

Y para convertir de **hexadecimal** a **decimal**:  $7 \cdot 16^0 + 10(A) \cdot 16^1 + 2 \cdot 16^2 = 679$ .

El programa debe tener un menú con 3 opciones:



Para esto debe hacer un programa con **2 funciones** (sino las hace, no tendrá validez el ejercicio):

**char\* DecToHex(int numDec):** esta función recibirá el número en decimal y deberá devolver un array dinámico con el número en hexadecimal.

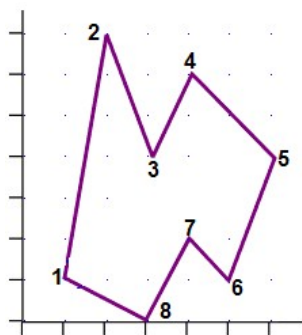
**int HexToDec(char\* numHex):** esta función recibirá el array con el numero en hexadecimal y deberá devolver un numero entero con el valor en decimal.

Por último, deberá hacer una función principal (**main**) que siga pidiendo los datos hasta que el usuario decida terminar el programa.

## 2.2. Polígonos irregulares con clases (2.5 pts.)

Se le ha pedido que haga un programa para una empresa de topografía, para calcular áreas y perímetros de parcelas, con puntos tomados con un GPS. Para ello necesita conocer el algoritmo del cálculo de un área de un polígono irregular y el perímetro.

El algoritmo del **cálculo del área (1,0 Pts.)** de un Polígono Irregular dice lo siguiente: Dados “N” puntos, ordenados en sentido horario o antihorario, el área del polígono es el llamado “producto en cruz” de todos los puntos, como se puede ver en la figura. Condición: que no corten el polígono. En la siguiente ecuación se encuentra dicho algoritmo:



x1 y1  
x2 y2  
x3 y3  
x4 y4  
...  
xn yn  
x1 y1

```

C:\WINDOWS\system32\cmd.exe
** Pol: Area y Perimetro **
Numero Vertices:75
x,y:71 1
x,y:71 2
x,y:72 3
x,y:73 2
x,y:73 1

** Puntos del Poligono **
[ 1, 1]
[ 1, 2]
[ 2, 3]
[ 3, 2]
[ 3, 1]

Perimetro :6.82843 m.
Area Total:3 m2.
Desea continuar(c) o salir(s):?s
Presione una tecla para continuar . . .
  
```

$$A(P) = \frac{1}{2} \cdot \left| \left( \sum_{i=0}^{i=n-2} (x_i \cdot y_{i+1} - y_i \cdot x_{i+1}) \right) + (x_{n-1} \cdot y_0 - y_{n-1} \cdot x_0) \right|$$

Para el **cálculo del perímetro (0,5 Pts.)**, ya sabe que es la suma de las distancias entre los vértices. Este algoritmo ya lo hemos realizado en varias ocasiones en las clases prácticas, por lo que no se describirá aquí. Aun así, si tiene dudas sobre el mismo, pregunte al profesor.

Para realizar este ejercicio, debe crear dos clases: la clase **Punto** y la clase **Poligono**. Vea a continuación la estructura de ambas clases:

```

class Punto
{
    double x,y;
public:
    Punto(double, double);
    void Prt();
    //agregar + funciones
};
  
```

```

class Poligono
{
    int n;          //Num de puntos total
    Punto *pv;      //Array de objetos Punto
public:
    Poligono(int n); //Const.
    void Add(Punto &p); //agrega punto
    double Area();    //calcula area
    double Perimetro(); //calcula Perimetro
    double Prt ();    //Imprime Puntos
    //agregar + funciones, si necesita
};
  
```

Desde la función principal **main()** debe pedir los datos: número de vértices y a continuación los valores (X,Y) de cada uno de ellos. Es muy fácil probar que ha hecho bien el programa, ya que puede introducir los vértices de un cuadrado y debe funcionar igual de bien. Al final del programa debe preguntar al usuario si continuar o salir. En caso de querer continuar, debe volver a empezar pidiendo los datos.

Por pantalla debe imprimir los datos de los Puntos dados (**0,5 Pts.**), mediante la función **Prt()** de la clase Polinono, la cual llamará a su vez a **Prt()** de la clase Punto.

**Grabar (0,25 pts):** En cada cálculo debe grabar los resultados (lo que se muestra en la pantalla, según la figura), en un fichero de texto, para lo cual debe pedir el nombre al usuario. Esto lo debe hacerlo por medio de una función **Grabar()**.

**Resto (0,25 pts.)**, se tendrá en cuenta la optimización del código, orden, comprobaciones, etc.

**NOTA: NO** puede cambiar las variables **privadas** a public. Si lo hace, el ejercicio se corregirá sobre la mitad de los puntos.

Si necesita agregar otras funciones a la clase, lo puede hacer.

## 2.3. Sistema eléctrico con herencia (3.5 pts.)

Se le pide programar las distintas opciones de un programa interactivo que gestiona un sistema eléctrico. Para ello se le proporciona a continuación la estructura principal del programa sobre la que tendrá que añadir su código. Si lo prefiere puede modificarla, y utilizar sentencias do-while, switch-case o lo que prefiera.

```
#include <cstdlib>
int Menu() {
    int opc = 0;
    while (opc < 1 || opc > 10) {
        system("cls"); // Limpia la pantalla de la consola
        cout << "MENU" << endl;
        cout << "\t1. Mostrar fecha con distintos formatos" << endl;
        cout << "\t2. Leer nombres de fichero y mostrar por pantalla" << endl;
        cout << "\t3. Ordenar nombres leídos y mostrar por pantalla" << endl;
        cout << "\t4. Añadir Central" << endl;
        cout << "\t5. Añadir mantenimiento" << endl;
        cout << "\t6. Mostrar informacion" << endl;
        cout << "\t7. Grabar datos en fichero" << endl;
        cout << "\t8. Ajustar potencia de una central" << endl;
        cout << "\t9. Ajustar potencia del sistema" << endl;
        cout << "\t10. Salir" << endl;
        cout << "    Opcion: ";
        cin >> opc;
    }
    return opc;
};

int Menu2() {
    int opc = 0;
    while (opc < 1 || opc > 2) {
        cout << "\nSUB MENU" << endl;
        cout << "\t1. Añadir central Eolica" << endl;
        cout << "\t2. Añadir central de Gas" << endl;
        cout << "    Opcion: ";
        cin >> opc;
        cout << endl;
    }
    return opc;
};

int Menu3() {
    int opc = 0;
    while (opc < 1 || opc > 3) {
```

```
    cout << "\nSUB MENU" << endl;
    cout << "\t1. Mostrar centrales Eolicas" << endl;
    cout << "\t2. Mostrar centrales de Gas" << endl;
    cout << "\t3. Mostrar todas las centrales" << endl;
    cout << "    Opcion: ";
    cin >> opc;
    cout << endl;
}
return opc;
};

vector<string> LeerDeFichero(ifstream& f);

void mostrarNombres(vector<string> names); //No es obligatoria, pero se recomienda
```

```
void main() {
    Eolica* listEolicas[100]; //Si lo desea puede utilizar vector<Eolica*>
    Gas* listGas[100];        //Si lo desea puede utilizar vector<Gas*>
    Planta* list[200];        //Si lo desea puede utilizar vector<Planta*>
    int conte = 0;            // Contador de centrales eólicas
    int contG = 0;            // Contador de centrales de gas
    int cont = 0;             // Contador de centrales
    int opc2=0, opc=0;
    bool flag = 0, flag2=0;
    vector<string> names;     // Vector con nombres disponibles para las centrales
    while (opc != 10){
        opc = Menu();
        if (opc == 1) {
            cout << "\nUso de namespaces: " << endl;
            int d, y, m;
            cout << "Día: "; cin >> d;
            cout << "Mes: "; cin >> m;
            cout << "Año: "; cin >> y;
            cout << "Fecha con formato europeo: ";
            // Añadir código necesario
            cout << "Fecha con formato internacional: ";
            // Añadir código necesario
            system("pause");
        }
        if (opc == 2) {
            cout << "\nNombres leídos de fichero: " << endl;
            // Añadir código necesario
            flag2 = 1;
            system("pause");
        }
        if (opc == 3 && flag2 == 1) {
            cout << "\nNombres ordenados alfabeticamente: " << endl;
            // Añadir código necesario
            system("pause");
        }
        if (opc == 4 && flag2 == 1) {
            cout << "\nAñadir central: " << endl;
            opc2 = Menu2();
            // Añadir código necesario
            system("pause");
        }
        if (opc == 5) {
            cout << "Añadir mantenimiento" << endl;
            // Añadir código necesario
            system("pause");
        }
        if (opc == 6) {
```

```
    cout << "\nMostrar informacion de centrales" << endl;
    // Añadir código necesario
    system("pause");
}
if (opc == 8) {
    cout << "\nAjustar potencia de una Central" << endl;
    // Añadir código necesario
    system("pause");
}
if (opc == 9){
    cout << "\nAjustar potencia del sistema" << endl;
    // Añadir código necesario
    system("pause");
}
if (opc == 10){
    cout << "\nSalir" << endl;
    // Añadir código necesario
}
}
}
```

A continuación, se le presenta el esqueleto de las clases y estructuras que va a utilizar en este programa.

```
// Añadir clase Fecha en un namespace

struct mantenimiento {
    Fecha fecha;
    string tarea;
};

class Central {
    string nombre_ID;
    double coste_base;           // Costes fijos de generación por MWh
protected:
    vector<double> potencia;     // En esta variable se almacena la potencia unitaria
                                // de cada elemento de la Central
    vector<double> carga;        // En esta variable se almacena la carga de cada elemento
                                // de la Central (0 = descativado, 1= maxima potencia)
public:
    vector<mantenimiento> historialMantenimiento;
public:
    Central(string _nombre_ID = "", double _coste_base = 0);
    void show_info();
    double CalcularPotenciaTotal();
    double CalcularPotenciaMaxima();
    bool AjustarCargaPotenciaObjetivo(double p);
    // GrabarEnFichero
    // Añada otras funciones si es necesario
};

class Eolica : public Central {
public:
    Eolica(string _nombre_ID = "", double _coste_base = 0);
    void AddTurbina(double p);
    // Añada otras funciones si es necesario
};

class EolicaOffshore : public Eolica {
public:
    EolicaOffshore(string _nombre_ID="", double _coste_base=0);
    // Añada otras funciones si es necesario
};

class Gas : public Central {
    double precio_gas;          // En € por tonelada
};
```



```
double consumo_gas; // En toneladas por MWh
public:
    Gas(string _nombre_ID = "", double _coste_base = 0, double _precio_gas=0, double
    _consumo_gas=0);
    void AddTurbinaGas(double p);
    // Añada otras funciones si es necesario
};
```

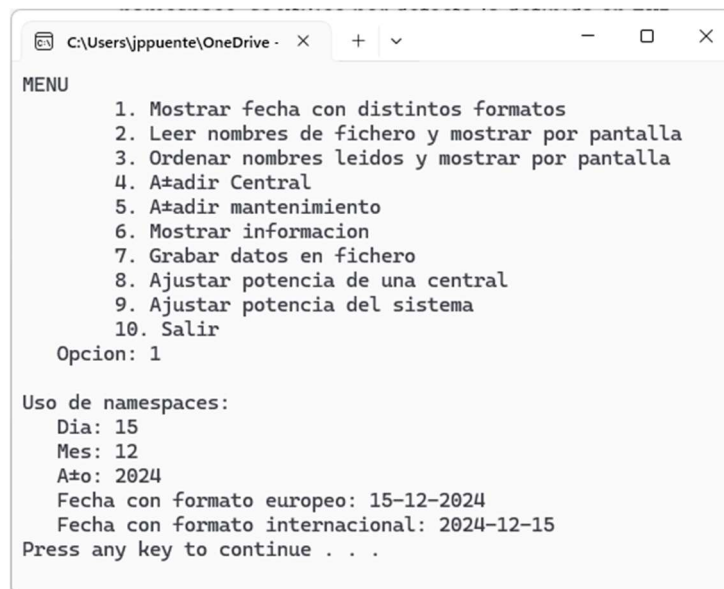
A continuación, se describe detalladamente lo que debe programar en cada opción

### 2.3.1. Opción 1: Mostrar fecha con distintos formatos

Deberá crear la clase `Fecha` en **dos namespaces diferentes**. El primero, denominado **EUR**, almacenará la fecha en formato europeo (**día/mes/año**). El segundo, denominado **Internacional**, deberá almacenar la fecha en formato internacional (**año/mes/día**).

En ambos casos, las variables **día, mes y año serán privadas**, y podrá emplear **el mismo constructor** para ambas clases si lo considera oportuno. Asimismo, cada clase deberá incluir una función `prt()` que muestre la fecha en el formato correspondiente a su namespace. Además, deberá incluir lo necesario para que, cuando se emplee la clase `Fecha` sin especificar namespace, se utilice por defecto la definida en **EUR**.

**Si no sabe utilizar namespaces**, podrá definir únicamente la clase `Fecha` con formato europeo de la manera habitual (sin utilizar namespaces). **En ese caso, se descontará la parte de la nota correspondiente al uso de namespaces**, pero podrá realizar las opciones 5, 6 y 7.



```
MENU
1. Mostrar fecha con distintos formatos
2. Leer nombres de fichero y mostrar por pantalla
3. Ordenar nombres leídos y mostrar por pantalla
4. A*adir Central
5. A*adir mantenimiento
6. Mostrar informacion
7. Grabar datos en fichero
8. Ajustar potencia de una central
9. Ajustar potencia del sistema
10. Salir
Opcion: 1

Uso de namespaces:
Dia: 15
Mes: 12
Año: 2024
Fecha con formato europeo: 15-12-2024
Fecha con formato internacional: 2024-12-15
Press any key to continue . . .
```

Figure 1 - Salida esperada de la opción 1

### 2.3.2. Opción 2: Leer nombres de fichero y mostrar por pantalla

En esta opción, el programa deberá **leer los nombres almacenados en el fichero** `Nombres.txt` y guardarlos en un vector de strings llamado `names`. Para ello, se deberá implementar la función `LeerDeFichero`, que tendrá como **entrada** un objeto de lectura de fichero y como **salida** un vector de strings con los nombres leídos.



Recuerde que puede utilizar la función `getline(istream, string)` para leer líneas completas, incluyendo aquellas que contengan espacios.

Una vez leídos los nombres, se deberán **mostrar por pantalla**. Aunque no es obligatorio, se recomienda crear una función `mostrarNombres` que reciba como **entrada** un vector de strings y no tenga **valor de salida**. Esta función facilitará la visualización y se podrá reutilizar en ejercicios posteriores. Puede ver la salida esperada por consola en la Figure 2. No se preocupe por el formato, en el ejemplo se muestra en dos columnas para que quede más compacto.

**Nota:** No se olvide de cerrar el archivo. Si no se sabe implementar la función `LeerDeFichero`, se permite **inicializar manualmente algunos nombres** en el vector `names` para poder continuar con el resto del ejercicio. En este caso, se descontará la parte correspondiente a la lectura desde fichero.

### 2.3.3. Opción 3: Ordenar nombres leídos y mostrar por pantalla

Esta opción permitirá **ordenar el vector** `names` y mostrar por pantalla los nombres **ordenados alfabéticamente**. Se recomienda reutilizar la función `mostrarNombres` creada en la opción 2 para visualizar los nombres de forma clara. Puede emplearse cualquier algoritmo de ordenación.

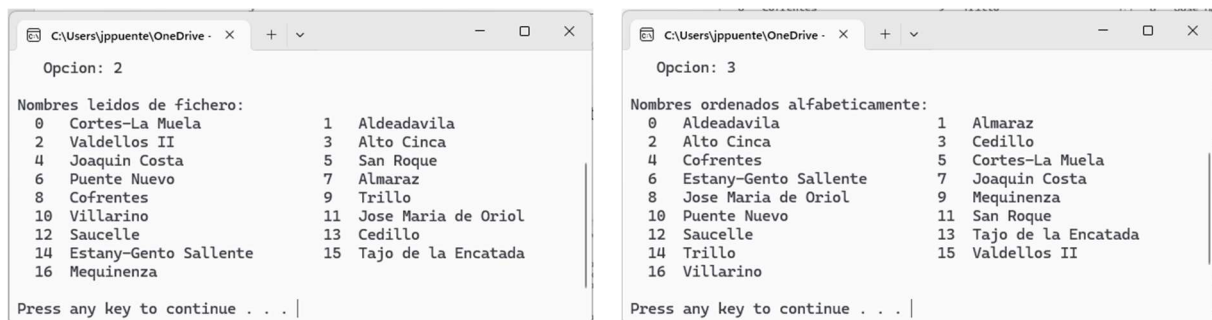
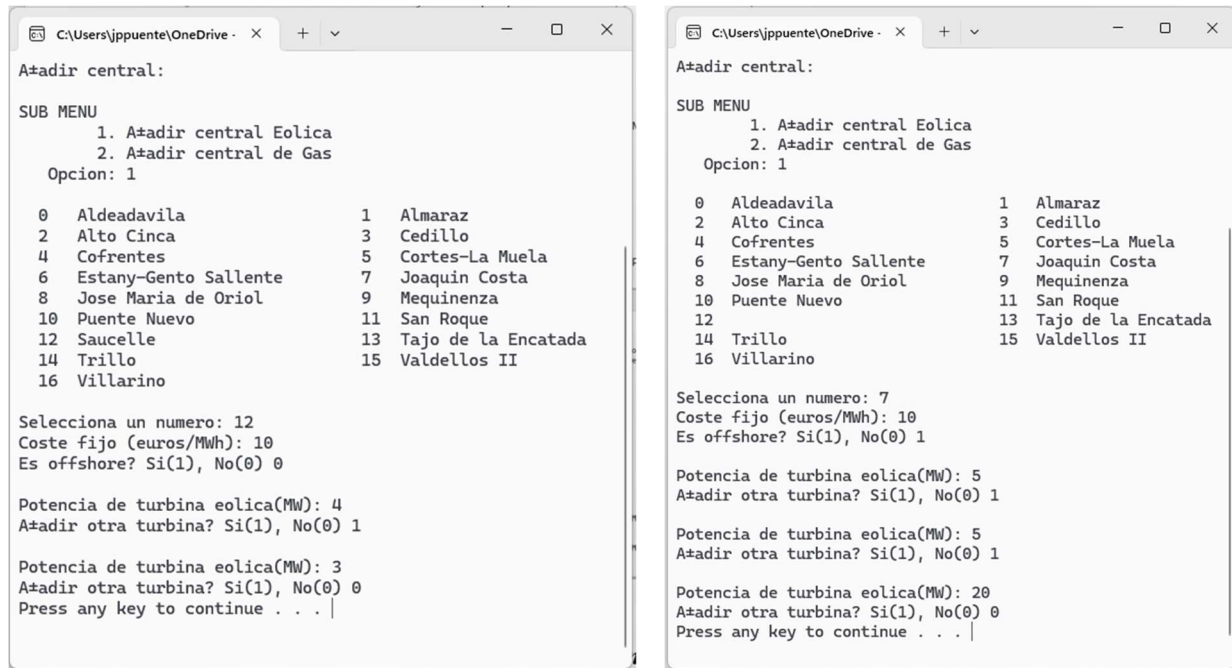


Figure 2 - Salida esperada de las opciones 2 y 3

### 2.3.4. Opción 4: Añadir central

Esta opción permite al usuario **añadir una nueva central eléctrica** al sistema. Primero, se pedirá que indique si desea crear una central **Eólica** o una central **de Gas**. A continuación, se mostrarán todos los nombres disponibles y el usuario deberá seleccionar uno para asignarlo a la nueva central.



```

C:\Users\jppuente\OneDrive - x + v - □ ×
A#adir central:

SUB MENU
  1. A#adir central Eolica
  2. A#adir central de Gas
Opcion: 1

0 Aldeadavila          1 Almaraz
2 Alto Cinca            3 Cedillo
4 Cofrentes            5 Cortes-La Muela
6 Estany-Gento Sallente 7 Joaquin Costa
8 Jose Maria de Oriol   9 Mequinenza
10 Puente Nuevo         11 San Roque
12 Saucelle             13 Tajo de la Encatada
14 Trillo               15 Valdellos II
16 Villarino

Selecciona un numero: 12
Coste fijo (euros/MWh): 10
Es offshore? Si(1), No(0) 0

Potencia de turbina eolica(MW): 4
A#adir otra turbina? Si(1), No(0) 1

Potencia de turbina eolica(MW): 3
A#adir otra turbina? Si(1), No(0) 0
Press any key to continue . . . |

C:\Users\jppuente\OneDrive - x + v - □ ×
A#adir central:

SUB MENU
  1. A#adir central Eolica
  2. A#adir central de Gas
Opcion: 1

0 Aldeadavila          1 Almaraz
2 Alto Cinca            3 Cedillo
4 Cofrentes            5 Cortes-La Muela
6 Estany-Gento Sallente 7 Joaquin Costa
8 Jose Maria de Oriol   9 Mequinenza
10 Puente Nuevo         11 San Roque
12 Saucelle             13 Tajo de la Encatada
14 Trillo               15 Valdellos II
16 Villarino

Selecciona un numero: 7
Coste fijo (euros/MWh): 10
Es offshore? Si(1), No(0) 1

Potencia de turbina eolica(MW): 5
A#adir otra turbina? Si(1), No(0) 1

Potencia de turbina eolica(MW): 5
A#adir otra turbina? Si(1), No(0) 1

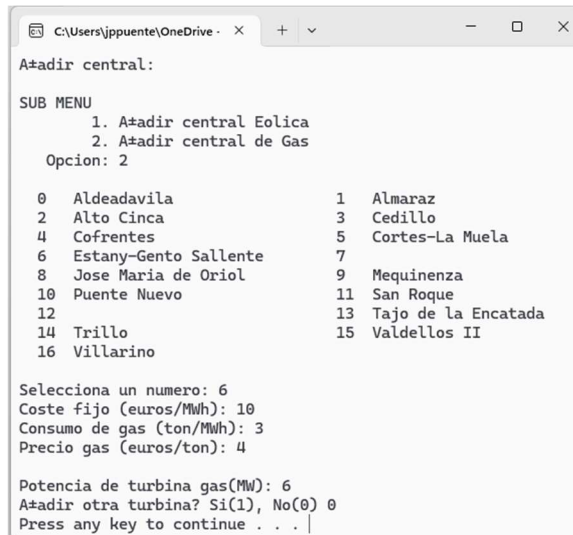
Potencia de turbina eolica(MW): 20
A#adir otra turbina? Si(1), No(0) 0
Press any key to continue . . . |

```

Figure 3 - Salida esperada de la opción 4 para el caso Eólica y EólicaOffshore

Si se elige una **central Eólica**, se solicitará el **coste fijo** de generación (euros/MWh) y se preguntará si es **Offshore**. Según la respuesta, se creará un objeto de tipo `Eolica` o `EolicaOffshore`, que se almacenará en la variable `listEolicas` (puede ser un array tal y como se define en el código que se le proporciona o un vector de punteros). Luego, se llamará a la función `AddTurbina` tantas veces como el usuario indique, **al menos una vez (ver imagen abajo)**. Cada llamada a esta función añadirá la potencia indicada al vector `potencia` y un cero al vector `carga`. Finalmente, el mismo puntero se copiará en la lista general de centrales `list`, para que quede registrado en ambos arrays (o vectores en el caso de que haya utilizado esa modalidad).

Si se elige una **central de Gas**, además del **coste fijo** de generación (euros/MWh), se pedirá el **consumo de gas** (ton/MWh) y el **precio del gas** (euros/ton). Se creará un objeto `Gas` con estos valores y se almacenará en la variable `listGas`. A continuación, se llamará a la función `AddTurbinaGas` tantas veces como el usuario indique (**al menos una vez**). Esta función agrega un elemento al vector `potencia` y un cero al vector `carga`. Al igual que en la central eólica, el puntero creado se copiará en la lista general `list`.



```

C:\Users\jppuente\OneDrive - x + v - □ ×
A#adir central:
SUB MENU
  1. A#adir central Eolica
  2. A#adir central de Gas
Opcion: 2

0 Aldeadavila          1 Almaraz
2 Alto Cinca           3 Cedillo
4 Cofrentes            5 Cortes-La Muela
6 Estany-Gento Sallente 7
8 Jose Maria de Oriol  9 Mequinenza
10 Puente Nuevo         11 San Roque
12                      13 Tajo de la Encatada
14 Trillo               15 Valdellos II
16 Villarino

Selecciona un numero: 6
Coste fijo (euros/MWh): 10
Consumo de gas (ton/MWh): 3
Precio gas (euros/ton): 4

Potencia de turbina gas(MW): 6
A#adir otra turbina? Si(1), No(0) 0
Press any key to continue . . .

```

Figure 4 - Salida esperada de la opción 4 para el caso Gas

Cada vez que se agregue una central deberá anteponer al nombre seleccionado la palabra “EOLICA”, “EOLICA OFFSHORE” o “GAS” en función del tipo de central. Una de las opciones es hacerlo a través del constructor, pero tiene libertad para hacerlo como desee.

Una vez terminado el proceso, **el nombre utilizado deberá eliminarse del vector** `names`. Puede hacerse utilizando la función `clear()` o sobrescribiendo el contenido con texto vacío.

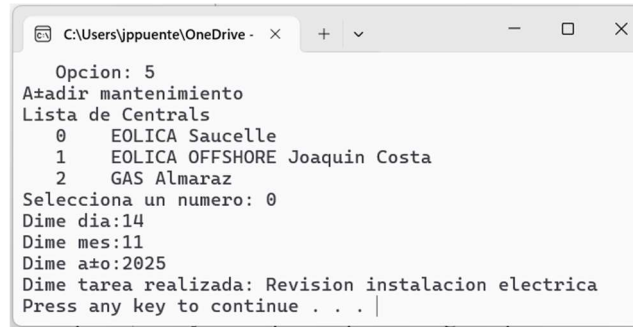
**Nota:** En todos los casos, el puntero del objeto recién creado se almacena **tanto en la lista específica del tipo de central** (`listEolicas` o `listGas`) como en la **lista general de centrales** `list`.

**Nota:** no es necesario añadir la comprobación de si ha seleccionado un nombre que ya no está disponible

### 2.3.5. Opción 5: Añadir mantenimiento

Esta opción permitirá al usuario **registrar una tarea de mantenimiento** en una de las centrales existentes. El programa deberá mostrar primero la **lista con los nombres de todas las centrales** disponibles y permitir al usuario seleccionar una de ellas. Una vez seleccionada, se solicitará al usuario la información del mantenimiento: día, mes, año y **descripción de la tarea realizada**. **Puede ver un ejemplo en la Figure 5**

Con estos datos, se deberá crea una estructura de tipo `mantenimiento`, asignando la fecha y la descripción introducida por el usuario, y agregarlo al vector de estructuras `mantenimiento` denominado `historialMantenimiento` de la central seleccionada.



```
Opcion: 5
A*adir mantenimiento
Lista de Centrales
0      EOLICA Saucelle
1      EOLICA OFFSHORE Joaquin Costa
2      GAS Almaraz
Selecciona un numero: 0
Dime dia:14
Dime mes:11
Dime a*o:2025
Dime tarea realizada: Revision instalacion electrica
Press any key to continue . . . |
```

Figure 5 - Salida esperada de la opción 5

### 2.3.6. Opción 6: Mostrar información

Esta opción permitirá al usuario **visualizar la información de las centrales eléctricas** del sistema. En el **submenú** el usuario puede elegir si mostrar: solo las **centrales eólicas**, solo las **centrales de gas** o todas las centrales disponibles. Para ello, deberá crear una función `show_info()` en la clase `Central` que muestre la siguiente información:

- **Nombre de la central.**
- **Potencia máxima instalada**, que es la suma de las potencias de todas las turbinas. Para ello implemente la función `CalcularPotenciaMaxima()` en la clase `Central`.
- **Precio por MWh** de la central mediante la función `CalcularPrecioMWh()`:
  - En centrales **Eólicas**, el coste es directamente el valor almacenado en `coste_base`
  - En centrales **EólicasOffshore**, el coste es `coste_base × 1.5`
  - En centrales de **Gas**, el coste es `coste_base + (precio_gas × consumo_gas)`
- **Potencia de cada turbina y su carga** expresada en porcentaje.
- **Historial de mantenimiento**, mostrando la fecha y la descripción de cada tarea (para esto es necesario haber implementado previamente las opciones 1 y 5).

Puede ver ejemplos de la salida esperada en la **Figure 6**

**Nota:** No se exige un formato exacto de salida. Si le sobra tiempo puede utilizar `setw()` (incluyendo `<iomanip>`) o configurar flags de `iostream` como `cout.setf(ios::left)` para mejorar la presentación de los datos.

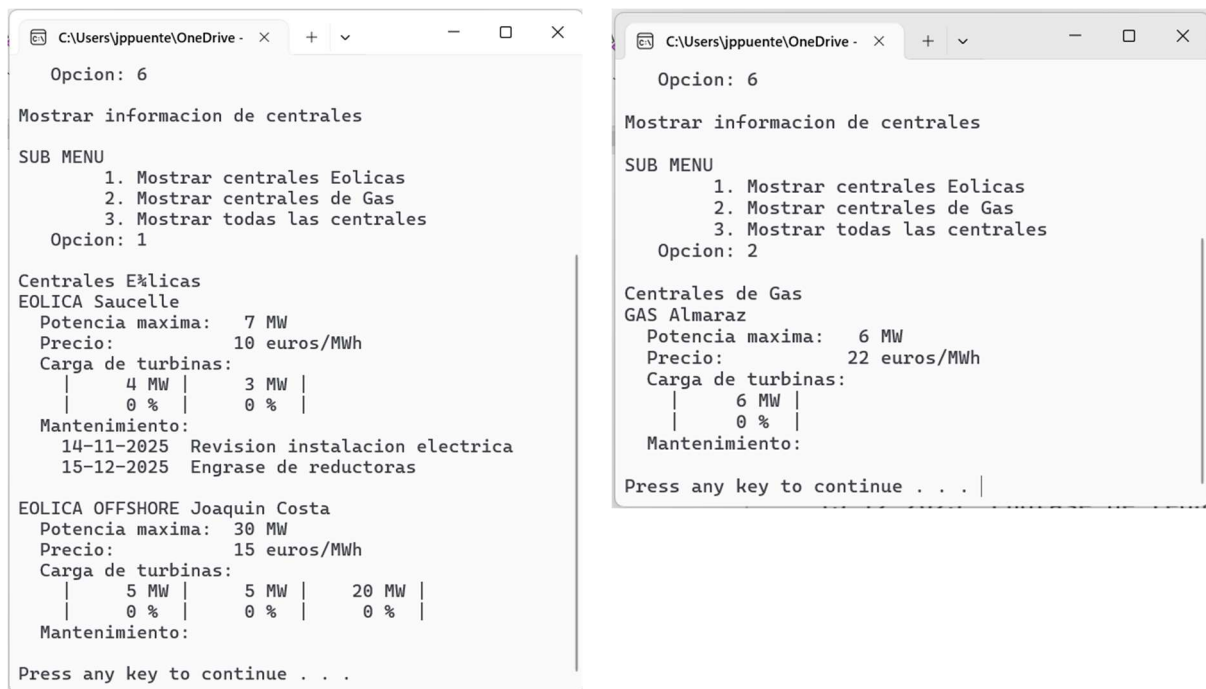


Figure 6 - Salida esperada de la opción 6 para el caso en el que se elija mostrar solo las centrales Eólicas o solo las centrales de Gas

### 2.3.7. Opción 7: Grabar datos en fichero

En esta opción, el programa deberá permitir al usuario **guardar la información de todas las centrales eléctricas en un fichero de texto**. El programa preguntará al usuario **el nombre del fichero** en el que desea guardar la información y creará un objeto de escritura en fichero.

Se deberá implementar en la clase Central una función llamada `GrabarEnFichero`, que reciba como **entrada** un objeto de escritura en fichero y **no devuelva variables de salida**. Esta función deberá **grabar la información exactamente en el mismo formato que la función `show_info()`**, de tal manera que se puede **copiar el código de `show_info()`, pegarlo y hacer las adaptaciones necesarias** para que escriba en el fichero en lugar de mostrar por pantalla.

La función `GrabarEnFichero` deberá incluir todos los datos relevantes de la central: nombre, potencia máxima, precio por MWh, potencia y carga de cada turbina, y el historial de mantenimiento con fechas y tareas realizadas.

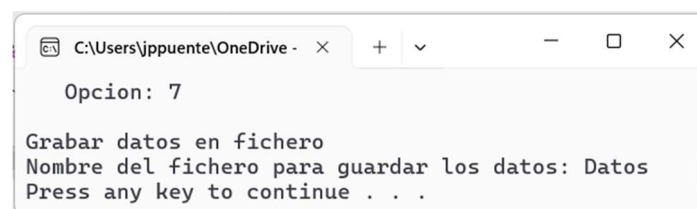


Figure 7 - Salida esperada de la opción 7

### 2.3.8. Opción 8: Ajustar potencia de una central

En esta opción, el programa permitirá al usuario **modificar la carga de una central específica** para alcanzar una potencia objetivo determinada.

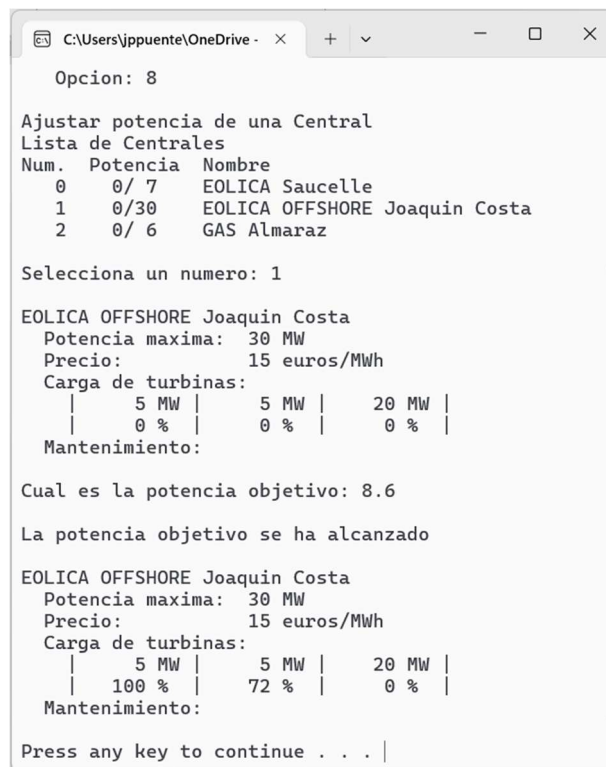
En primer lugar, se mostrarán todas las centrales existentes indicando, para cada una de ellas, su **potencia actual**, **potencia máxima** y **nombre**, y se pedirá al usuario que seleccione una de ellas (ver Figure 8). Para ello deberá programar la función `CalcularPotenciaTotal()`, que devuelve la potencia actual, calculada como el sumatorio del producto entre la potencia y la carga de cada turbina. Para implementarla, puede copiar el código de la función `CalcularPotenciaMaxima()` y adaptarlo para que tenga en cuenta la carga de cada turbina:

$$P_{Tot} = \sum_i P_i \cdot C_i$$

Una vez seleccionada la central, se mostrará toda su información utilizando la función `show_info()` y se preguntará al usuario cuál es la **potencia objetivo** que desea alcanzar. Para realizar el ajuste de la carga, se recomienda programar la función `AjustarCargaPotenciaObjetivo(double p)` en la clase `Central`. Esta función recibe como parámetro la potencia objetivo; devuelve un valor booleano indicando si ha sido posible realizar el ajuste; y modifica internamente el vector `carga` para que la potencia total (`CalcularPotenciaTotal()`) se ajuste a la potencia objetivo.

Para implementar el ajuste, se recomienda **poner inicialmente todas las cargas a cero** y, posteriormente, **ir aumentando progresivamente la carga de cada turbina** hasta alcanzar la potencia deseada. Si la potencia objetivo se puede alcanzar con las turbinas existentes, la función devolverá `true`; en caso contrario devolverá `false`.

Tras realizar el ajuste, deberá mostrarse nuevamente la información de la central mediante la función `show_info()` para que el usuario pueda verificar que los cambios se han aplicado correctamente. **Puede ver ejemplos de la salida esperada en la Figure 8.**



```

Opcion: 8

Ajustar potencia de una Central
Lista de Centrales
Num.  Potencia  Nombre
0      0/ 7      EOLICA Saucelle
1      0/30      EOLICA OFFSHORE Joaquin Costa
2      0/ 6      GAS Almaraz

Selecciona un numero: 1

EOLICA OFFSHORE Joaquin Costa
Potencia maxima: 30 MW
Precio:          15 euros/MWh
Carga de turbinas:
|    5 MW |    5 MW |    20 MW |
|    0 %  |    0 %  |    0 %  |
Mantenimiento:

Cual es la potencia objetivo: 8.6

La potencia objetivo se ha alcanzado

EOLICA OFFSHORE Joaquin Costa
Potencia maxima: 30 MW
Precio:          15 euros/MWh
Carga de turbinas:
|    5 MW |    5 MW |    20 MW |
|   100 % |    72 % |    0 %  |
Mantenimiento:

Press any key to continue . . .

```

Figure 8 - Salida esperada de la opción 8



### 2.3.9. Opción 9: Ajustar potencia del sistema

En esta opción, el programa permitirá **ajustar de manera automática** la potencia generada por todas las centrales del sistema con el objetivo de alcanzar una potencia global solicitada por el usuario. Para comenzar, el programa **mostrará**, para cada central existente, su potencia actual, su potencia máxima y su nombre. Después de esta visualización, **solicitará al usuario** que introduzca la potencia objetivo que desea alcanzar en el conjunto del sistema (ver Figure 9).

Una vez introducida la potencia objetivo, el sistema realizará el ajuste en dos fases. En la primera, **establecerá la carga de todas las centrales a cero**, dejando así la potencia generada por todas las centrales (`CalcularPotenciaTotal()`) inicialmente en cero. En la segunda fase se procederá a **repartir la potencia solicitada entre las distintas centrales siguiendo el orden en el que se encuentran almacenadas** (se hace de manera análoga a la opción 8). Para cada central, siempre que aún quede potencia por cubrir, se llamará a la función `AjustarCargaPotenciaObjetivo` pasando como argumento la potencia restante. Si la potencia que queda por asignar **supera la potencia máxima disponible** en dicha central, esta se ajustará a su capacidad máxima y el resto de potencia pendiente se intentará cubrir utilizando las centrales posteriores. En cambio, si la central puede cubrir completamente la potencia pendiente, se ajustará exactamente a la potencia solicitada y la potencia restante pasará a ser cero.

Una vez completado este proceso, el programa determinará si la potencia objetivo ha podido alcanzarse con las centrales disponibles. **Si ha sido posible**, mostrará un mensaje indicando que la potencia del sistema se ha ajustado correctamente (ver Figure 9 derecha). **En caso contrario**, indicará que la potencia instalada no ha sido suficiente y que el sistema se ha ajustado a la máxima potencia posible (ver Figure 9 izquierda). Para finalizar, se mostrará de nuevo la potencia actual y la potencia máxima de cada central, permitiendo verificar visualmente el resultado del ajuste. En la Figure 10 puede ver cómo quedan las cargas de cada central (se ejecuta la opción 6) después de haber ejecutado esta función.

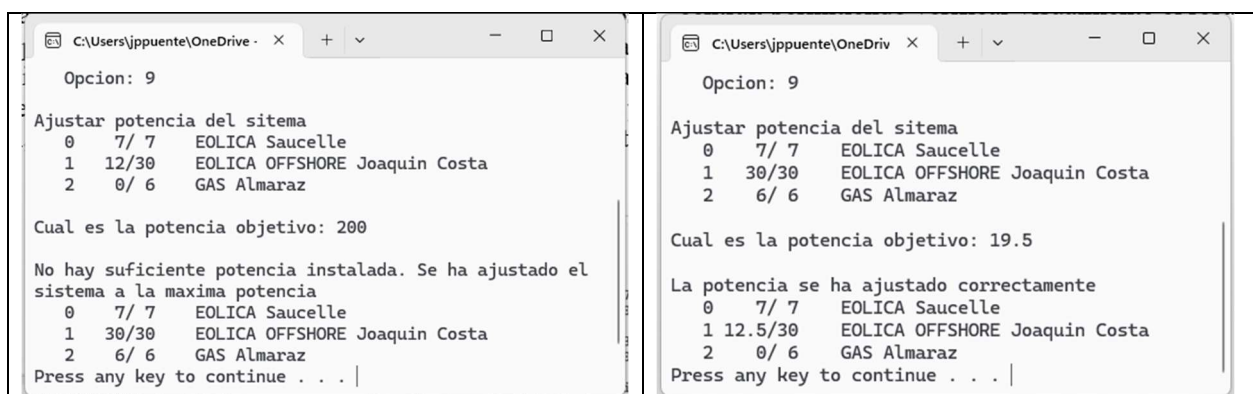
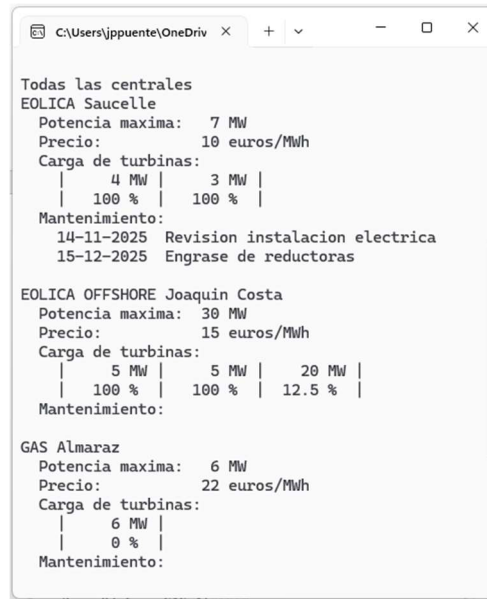


Figure 9 - Salida esperada de la opción 9 para el caso en el que no es posible ajustar la potencia máxima y para el caso en el que sí es posible ajustarla





```
C:\Users\jppuente\OneDrive
Todas las centrales
EOLICA Saucelle
Potencia maxima: 7 MW
Precio: 10 euros/MWh
Carga de turbinas:
| 4 MW | 3 MW |
| 100 % | 100 % |
Mantenimiento:
14-11-2025 Revision instalacion electrica
15-12-2025 Engrase de reductoras

EOLICA OFFSHORE Joaquin Costa
Potencia maxima: 30 MW
Precio: 15 euros/MWh
Carga de turbinas:
| 5 MW | 5 MW | 20 MW |
| 100 % | 100 % | 12.5 % |
Mantenimiento:

GAS Almaraz
Potencia maxima: 6 MW
Precio: 22 euros/MWh
Carga de turbinas:
| 6 MW |
| 0 % |
Mantenimiento:
```

Figure 10 - Salida esperada tras ejecutar la opción 6 después de haber hecho el ajuste de potencia del sistema

### 2.3.10.Opción 10: Salir

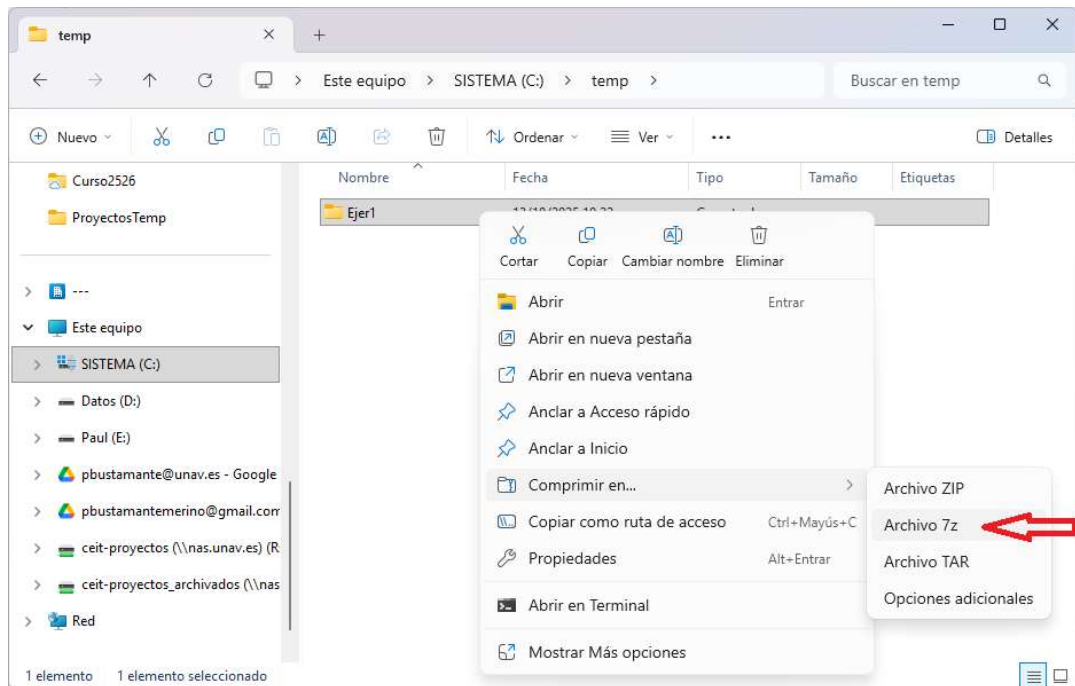
Al ejecutar esta opción terminará el programa. Recuerde que debe liberar la memoria dinámica que hay reservado.

**NOTA:** Deberá realizar lo que se pide en todos los ejercicios: funciones, reserva dinámica de memoria, etc. Caso contrario, no se puntuarán dichos ejercicios.

### 3. ANEXO A: COMPRIMIR CARPETAS

Los pasos para comprimir las carpetas de los ejercicios del examen son:

- Seleccionar la carpeta con el **botón derecho** del ratón y darle a la opción de “Comprimir en..” y “Archivo 7z”.



- Debe de haber generado el fichero “Ejer1.7z” en C:\Temp

